Low Powered Automatic Self Water Plant Pot

Kasper Bloch Mathiasen
Author
SDU
Odense, Denmark
kmath22@student.sdu.dk

Anders Stengaard
Supervisor
SDU
Odense, Denmmark
maedje@typst.app

Abstract*—Maintaining plants in offices can be challenging during weekends and vacations when regular watering is not possible. This project presents the development of a low-power, low-cost automatic self-watering plant pot designed to operate for extended periods without external power. The system uses a Raspberry Pi Pico, a soil moisture sensor, a water pump, and an e-ink display. To minimize power consumption, the Pico enters dormant mode between watering cycles and is woken approximately every six hours using an external binary counter. A Run-To-Complete scheduler manages the system tasks, while a fuzzy controller is used to determine watering duration based on soil moisture measurements. The implemented components were successfully tested together on a breadboard, demonstrating the basic functionality of the system. Theoretical calculations estimate an average current consumption of 0.845 mA, corresponding to approximately three months of operation using an 1800 mAh battery. However, the estimated battery lifetime could not be experimentally verified due to damage to the battery shield during implementationMaintaining plants in offices can be challenging during weekends and vacations when regular watering is not possible. This project presents the development of a low-power, low-cost automatic self-watering plant pot designed to operate for extended periods without external power. The system uses a Raspberry Pi Pico, a soil moisture sensor, a water pump, and an e-ink display. To minimize power consumption, the Pico enters dormant mode between watering cycles and is woken approximately every six hours using an external binary counter. A Run-To-Complete scheduler manages the system tasks, while a fuzzy controller is used to determine watering duration based on soil moisture measurements. The implemented components were successfully tested together on a breadboard, demonstrating the basic functionality of the system. Theoretical calculations estimate an average current consumption of $0.845[\mathrm{mA}]$, corresponding to approximately three months of operation using an $1800[\mathrm{mAh}]$. However, the estimated battery lifetime could not be experimentally verified due to damage to the battery shield during implementation. Link to the GitHub can be found here

I. Introduction

We spend many hours in our offices at out workplace, this can have a negative impact on our mental health. A way to combat this is to set up plants around the office to help improve the overall mental health in that room [1]. One such plant could be a tomato plant, as it can help improve the overall mental health along with supplying a healthy snack in the form of tomatoes. However, one downside to these plants is the need to be watered daily, which can prove difficult as most office workers are not able to water them during the weekends or vacations. This could result in a work environment getting worse due to dead or rotting plants being in the office. In this individual self study project a low power, low budget self watering plant pot is developed to enable autonomous care of plants. The pot should be battery powered to ensure it is not restricted by power outlets and should be low powered and able to run without being charged for an extended period of time.

II. Related Works

Many such self watering plant pots can be bought from various online stores, such as this growing system [2] that needs to be plugged into the wall, or this more expensive smart plant pot [3] that has an integrated temperature sensor and moisture sensor. The Smart Pot does not state the battery life but is most likely not multiple months. There also exists many mechanical(passive) self watering plant pots, but many share the same limitation: The roots are constantly moist which restricts oxygen and can lead to root rot[4], while some watering systems transport water too slowly for some plants, such as tomatoes[5].

To address some of these limitations, this project focuses on developing a low-power, battery-operated self-watering system that uses soil moisture measurements to control the amount of water supplied to the plant. This allows watering to be adjusted based on the measured soil conditions while aiming to maintain a long battery life.

III. Methods

To ensure long battery life, low powered components are used in the pot. The main component of the pot is a Pico Pi running a Run-To-Complete scheduler and using dormant mode to ensure a low as possible power draw when the system is idling [6]. Thus the components used should be low power and low budget off the shelves components.

A. Components

The components used in this project is:

- HW-101 HW-moisture sensor V1.2.
- SC3101PW Vacuum diaphragm pump 3V [7].
- Binary Counter [8].
- Pico Pi 2W.
- Rechargeable Lithium Battery $1800[\mathrm{mAh}]$.
- Battery Shield Wemos D1 mini.
- Motor Driver TB6612FNG.
- SD card module(planned, not integrated in hand in prototype).
- Eink Display 2.9 waveshare.

In order to use the pico pi in dormant mode an external wakeup call is needed, as the dormant mode disable the pico's internal clock, making it unable to wake it self. To overcome this, a binary counter is used, as with the correct resistor and capacitance used with it results in the output signal going high approximately every 6 hours. This signal will then be used as the external wakeup signal for the pico pi. To avoid the counter staying high for 6 hours as well, the pico resets the counter before going to deep sleep again. This will allow the pico to draw $0.8[mA]$ [6] as it enters dormant mode.

The pump used is a cheap $3[\mathrm{V}]$ vacuum pump that draws $65[\mathrm{mA}]$ to $170[\mathrm{mA}]$ and will be driven with a TB6612FNG motor controller. With the pump running only at very short bursts, this should fit well in the project.

A $1800[\mathrm{mAh}]$ rechargeable lithium battery is used together with a battery shield to allow for easy charging using a micro USB cable.

To enable some sort of user interface an Eink display is used to display moisture, and watering time. Eink displays are a perfect fit for this type of project as they only use power when updating the display, after which they use no power yet still show the same image.

B. Software

The software for the Pico Pi is written in C to ensure the program runs efficient when the Pico is awake. The software is build around a simple Run To Complete(RTC) scheduler to make developing modular and accessible. The functionality of the pot is divided into tasks as seen in Fig. 1. The first task after wakeup is to measure the moisture in the soil and write the result to the soil struct. The software is designed to support logging moisture measurements and watering amounts to an SD card. However, this functionality was not implemented in the final prototype due to a missing connector on the PCB and is therefore left for future development. After reading the moisture sensor output the value is inserted into a simple membership function to determine the linguistic value for the linguistic variable moisture. This is inserted into a fuzzy rule set to determine the value for watering time. Using this watering time value, the pump runs for the given amount of time. The final task before dormant mode is updating the Eink display with moisture level and watering amount.

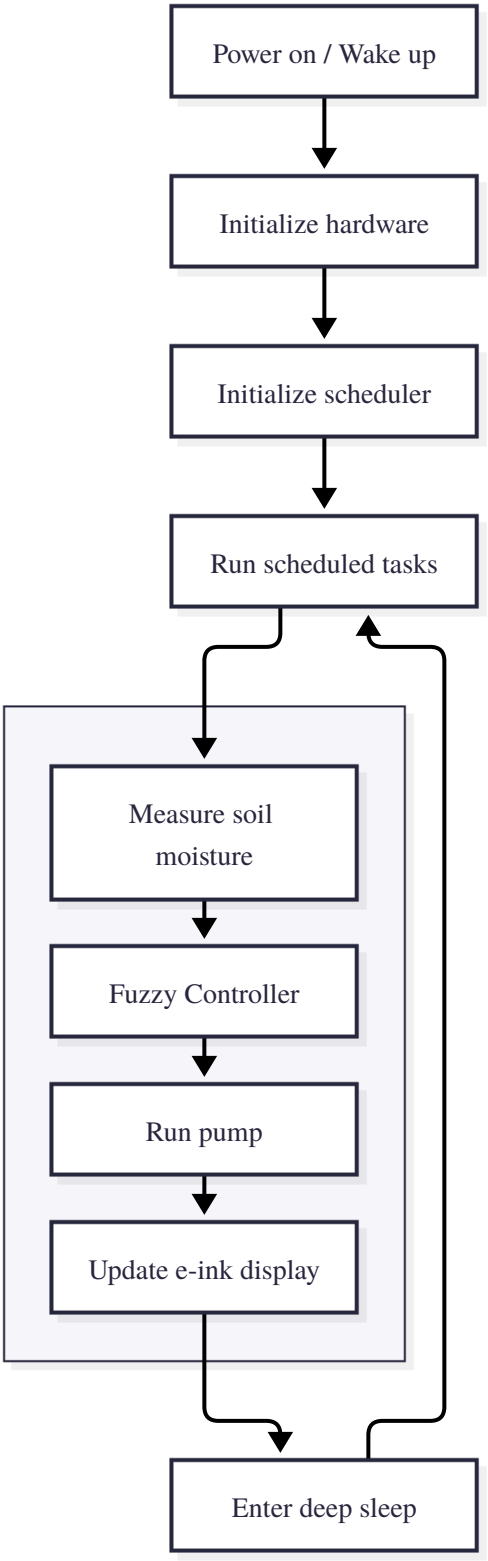


Fig. 1. Software Diagram

IV. Hardware

A. PCB Design

A PCB was designed in KiCAD to house all the components. The PCB design was sent to Aisler for manufacturing and was then soldered manually after receiving it. Small mistakes where made on the pcb design: Forgetting to route a wire for on/off for the moisture sensor, using the wrong footprint for the 4060E and forgetting to add a connector for an sd card. Though these mistakes did not invalidate the whole pcb. The 3D design can be seen in Fig. 2.

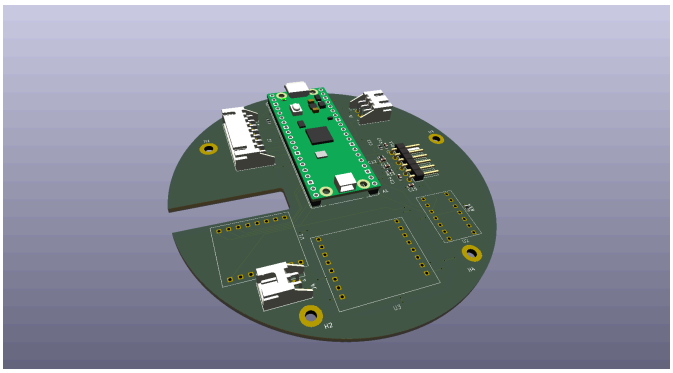


Fig. 2. PCB Design

B. Dormant Mode

To ensure the pi sleeps for close to 6 hours, the 4060E is used as an external wake up signal. The oscillating frequency is determined by:

$\frac{1}{2.2RC} = \frac{1}{2.2 \cdot 1.2[\text{M}\omega] \cdot 1[[\mu\text{F}])} = 0.38[\text{Hz}]$

And using output Q14 gives a frequency of:

$Q_{14} = \frac{0.38}{16384} = 0.000023[\text{Hz}]$

This gives approximately a 12 hour cycle, though it should be mentioned that the $1.2[\text{M}\omega]$ resistor is outside the specified resistance range of the 4060E, so the 6 hour low time will not be accurate.

C. Battery Life

a) *4060E:*

The dynamic power consumption is determined by the formula from the datasheet[8]:

$P_D = C_{PD} \cdot V_{CC}^2 f_i + \sum\left(C_L V_{CC}^2 \frac{f_i}{M}\right)$

where $M = 2^1, 2^2, 2^3 ... 2^{14}$a, f_i =input frequency, C_L = output load capacitance.

Inserting the values from our circuit:

$C_L = 1[\text{pF}]$, $R = 1.2[\text{M}\omega]$, $C = 1[\mu\text{ F}]$, $C_{PD} = 40[\text{pF}]$, $V_{CC} = 3.3[\text{V}]$.

With a frequency of $0.38[\text{Hz}]$ and the output load of the pico pin is omitted in the dynamic formula:

$40[\text{pF}] \cdot (3.3[\text{V}])^2 \cdot 0.38[\text{Hz}] = 0.166[\text{nW}]$

Thus the power draw of the 4060E when adding static and dynamic current draw is approximately $\frac{0.166[\text{nW}]}{3.3[\text{V}]} = 0.050[\text{nA}] + 8[\mu\text{ A}] \approx 8[\mu\text{ A}]$

b) *Pump and Controller:*

The max standby input current of the motor controller is rated at $1[\mu\text{ A}]$.

The pump max operating current draw is rated at $170[\text{mA}]$ [7]. In a test setup the pump was measured to draw approximately $100[\text{mA}]$ under light load. Thus $100[\text{mA}]$ will be used as a mean throughout the calculations.

c) *Moisture sensor:*

The rated operating current of the moisture sensor is $5[\text{mA}]$ [9]. The sensor will be turned of while in dormant mode as it is not necessary to be active while during that period.

d) *Eink Display:*

The eink display draws $\frac{26.4[\text{mW}]}{3.3[\text{V}]} \approx 8[\text{mA}]$ when refreshing over 3 seconds and $\approx 0.01[\mu\text{A}]$ on standby [10].

e) *Pico Pi:*

The mean dormant current draw of the pico pi is $0.8[\text{mA}]$ according to the datasheet [6].

f) *Dormant Current Draw:*

Adding the standby/dormant currents up to get the total current draw when dormant:

$8[\mu\text{ A}] + 1[\mu\text{ A}] + 0.01[\mu\text{ A}] + 0.8[mA] = 0.809[\text{mA}]$.

Thus a total dormant current draw of $0.809[\text{mA}]$ during dormant mode.

g) *Active Current Draw:*

Once active the pump phase draws:

$I_{\text{Pump Phase}} = I_{\text{Pump}} + I_{\text{Pico}} + I_{4060} = 100[\text{mA}] + 25[\text{mA}] + 8[\mu\text{ A}] \approx 125.008[\text{mA}]$

For a maximum of 5 seconds, resulting in a maximum per pump current draw of:

$Q_{\text{Motor Phase}} = 125.008[\text{mA}] \cdot \frac{5}{3600} = 0.17362[\text{mAh}]$

With the moisture sensor running 2 seconds:

$Q_{\text{Sensor Phase}} = (5[\text{mA}] + 25[\text{mA}] + 8[\mu\text{ A}l]) \cdot \frac{2}{3600} \approx 0.01667[\text{mAh}]$

The eink refreshes for 3 seconds drawing $\frac{26.4[\text{mW}]}{3.3[\text{V}]} = 8[\text{mA}]$ resulting in a current draw of:

$Q_{\text{Eink Pase}} = (8[\text{mA}] + 25[mA] + 8[\mu\text{ A}]) \cdot \frac{3}{3600} = 0.02751[\text{mAh}]$

The phases are separated as each phase is implemented in software as a task, all running consecutively. Thus, adding all the active phases together:

$Q_{\text{active}} = Q_{\text{Motor Phase}} + Q_{\text{Sensor Phase}} + Q_{\text{Eink Phase}} = 0.17362[\text{mAh}] + 0.01667[\text{mAh}] + 0.02751[\text{mAh}] = 0.21780[\text{mAh}]$

h) *Total Current Draw:*

A full cycle of 6 hours dormant and one active cycle of 10 seconds totals the current draw at:

Dormant period without active time: $t_{\text{dormant}} = 21600[\text{s}] - 10[\text{s}] = 21590[\text{s}]$

Assumed dormant current becomes:

$0.80901[\text{mA}] \cdot \frac{21590[\text{s}]}{3600[\text{s}]} = 4.85181[\text{mAh}]$

Adding the active current draw:

$4.85181[\text{mAh}] + 0.21780[\text{mAh}] = 5.06961[\text{mAh}]$

Meaning a full 6 hour cycle draws $5.06961[\text{mAh}]$

This means that the estimated total currant draw is:

$\frac{5.06961[\text{mAh}]}{6[\text{h}]} = 0.84494[\text{mA}]$, this can be seen illustrated in Fig. 3.

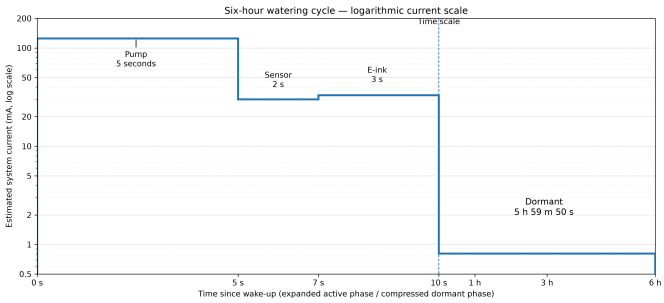


Fig. 3. Estimated Current Consumption

Meaning that the total battery life will be:
$\frac{1800[\text{mAh}]}{0.84494[\text{mA}]} = 2130[\text{h}]$
or:
$\frac{\frac{2130[\text{h}]}{24}}{30} = 2.96[\text{months}]$. Note that this is a theoretical optimistic estimate and based on numbers from the data sheets.

V. Test

To determine if the estimated power consumption is correct the circuit will be tested by connecting the battery shield to a power supply with a multimeter in series in between. With this configuration the total current draw of both dormant and active mode can be read on the multimeter. Unfortunately the battery shield was damaged during implementation and can no longer be used, thus the test could not be performed in time for the project hand in.

All components was however tested in a breadboard and proved the functionality of the whole system.

VI. Results

All components worked during testing in a breadboard and showed all the components working as expected in unison. The estimated battery life is 3 months derived from the calculated total current draw of $0.85[\text{mA}]$ on a $1800[\text{mAh}]$ lithium battery. Real life measurements was not done in time as the battery shield was damaged when implementation the components in the pcb, hindering the real life tests.

VII. Learning Achievements

A. PCB Design

In this project i designed, ordered and soldered my first PCB. The pcb was manufactured professionally by the company Aisler. The pcb was functioning correctly but had some mistakes such as forgetting to implement a connector for an SD card and routing a wire for on/off for the moisture sensor. Though these mistakes will be fixed in the GitHub repo.

B. Low Power Embedded Projects

Through this project i went through the full pipeline of creating a low powered embedded system. Starting out was choosing the right components that fit the low powered criteria, such as using an eink display for UI and a 4060E for a low powered wake signal. Another challenge that was overcome was to fit all the components together in software and creating a simple efficient control loop.

C. Mistakes

The ordered battery shield module has outputs 5V, GND and 3.3V, but it seems that the module was damaged during implementation and now outputs 10V on the 5V pin and 0V on the 3.3V pin. This is likely due to a short circuit when inverting the encoder poles to ensure the encoder switch is active HIGH and not active LOW.

VIII. Future Work

The project will be made public on GitHub and will continue to be updated and improved upon. Firstly, the mistakes made on the hardware will be fixed and a new PCB will be designed to correct the footprint mistakes. A new battery shield will be ordered to replace the broken one.

After the errors has been fixed, a fuel gage will be added to measure the battery percentage which will be displayed on the eink display. An SD card module will also be added to enable logging during operation and can enable plotting of various data on the eink display.

IX. Conclusion

In this project, a low-power automatic self-watering plant pot was developed using a Raspberry Pi Pico, a soil moisture sensor, a water pump, and an e-ink display. To reduce power consumption, the Pico uses dormant mode together with an external binary counter to wake the system approximately every six hours. The software was developed using a simple Run-To-Complete scheduler to manage the different tasks of the system.

The implemented components was successfully tested together on a breadboard, demonstrating the basic functionality of the self-watering system. Based on theoretical calculations using component datasheets, the system has an estimated average current consumption of $0.845[\text{mA}]$, resulting in a battery life of approximately three months with an $1800[\text{mAh}]$ battery. However, this estimate could not be verified through physical measurements due to damage to the battery shield during implementation.

Although the project encountered some hardware design mistakes, it provided valuable experience in designing PCBs, selecting low-power components, and developing embedded software. Further improvements are needed before the system can be considered fully completed, including correcting the PCB design, implementing SD card logging, and verifying the actual power consumption.

X. Acknowlegdement

LLM's was used in the process or correcting the report.

The plant pot CAD design is derived from the work of Martin [11].

References

[1] C. Hall and M. Knuth, "An update of the literature supporting the well-being benefits of plants: A review of the emotional and mental health benefits of plants," *Journal of Environmental Horticulture*, vol. 37, no. 1, pp. 30–38, 2019.

[2] "Indoor Garden Hydroponics Growing System," [Online]. Available: https://www.amazon.com/Indoor-Garden-Hydroponic-Growing-System/dp/B0B6BB4TVC

[3] "Pink Smart Plant Pot with SelfWatering Planter," [Online]. Available: https://www.fruugo.dk/lyserod-smart-plantepotte-med-selvvandende-plantepotte-fugtighedssensor-og-temperaturovervagning-til-indendorsbrug/p-452581288-954266539?language=da&ac=bing&__cf_chl_tk=..d.x79.bofFx8IkzlLHI_DMLydRCv2S.zhYkMPNymg-1793200571-1.0.1.1-foSsp2BF_7zFkhlPEG5Lil2ro7Sqhve1scGjv6MMXDA

[4] "Root Rot," [Online]. Available: https://extension.wsu.edu/pnw-gardeners-handbook/chapter-14-houseplants/

[5] "A Semi-Systematic Review of Capillary Irrigation," [Online]. Available: https://www.mdpi.com/2311-7524/4/3/23

[6] "Pico Pi dormant mode++: Pico pi documentation," [Online]. Available: https://pip-assets.raspberrypi.com/categories/610-raspberry-pi-pico/documents/RP-008307-DS-2-pico-datasheet.pdf

[7] "Vacum pump," [Online]. Available: https://www.laskakit.cz/en/sc3101pw-vakuove-membranove-cerpadlo-3v/

[8] "Binary Counter," [Online]. Available: https://www.digikey.dk/en/products/detail/texas-instruments/SN74HC4060DBR/1571132

[9] "Moisture sensor Datasheet," [Online]. Available: https://docs.sunfounder.com/projects/esp32-starter-kit/en/latest/components/component_moisture_module.html

[10] "Eink Display," [Online]. Available: https://www.waveshare.com/2.9inch-e-paper-module.htm

[11] "Plant pot inspiration," [Online]. Available: https://www.thingiverse.com/thing:4921885